

Prog2-nyilvántartás

Nagy házi terv - Egyházi Zoltán Tamás

Table of Contents

Választott feladata:	1
Röviden a programról:	1
Adatok beolvasása indításkor:	3
Főmenü:	4
Listázás:	5
Megvalósítás:	7
Adatszerkezet:	7
Oktatói rekord - Változó számú csoporttal:	7
Hallgatói rekord - Változó számú ZH pontszámmal:	7
Objektumorientált modell:	8
Szemely (Absztrakt őssztály):	8
Hallgato (Szemely osztályból származtatott):	8
Oktato (Szemely osztályból származtatott):	8
Parser (Adatkezelő motor):	9
Menu (Vezérlő interfész):	9
Algoritmusok:	10
1. Dinamikus tömb bővítése (Parser::atmeretez):	10
2. Rugalmas sorfeldolgozás (Parser::sorFeldolgoz):	10
3. Polimorf listázás (Parser::listazMindenkit):	11

Választott feladata:

A prog1-es **nyilvántartás** feladatot szeretném megvalósítani újra **OO** szemlélettel.

Ez az alábbi volt: **Egy egyszerű nyilvántartó program** a hallgatók és oktatók adatainak tárolására, valamint az **eredményeik megjelenítésére**.

Röviden a programról:

A program **konzolos alkalmazás**.

Szöveges **menürendszer**t használ amin **számok** beküldésével lehet **navigálni**.

Indításkor beolvassa a **hallgatókat** és oktatókat **tároló** fájlokat.

A program indítás után a **konzolon** egy **szöveges főmenüt** jelenít meg, innen érhetők el a program egyes **funkciói**. Ezek a következők:

- **Listázás:** a program által nyilvántartott adatok szűrt/szűrés nélküli listázása a kijelzőre.
- **Új adat felvétele:** a felhasználó által választott, az előre megadott opciók közül választva, és az általa megadott adatok felvétele.

- **Információk szerkesztése:** már létező adatok név alapján történő kiválasztása, majd szerkesztése.
- **Kilépés a programból**

A fejlesztéshez kívánom felhasználni az prog1-es megvalósításnak a kódbázisát, hisz egyes elemei nyelv függetlenek, például a menük és kiírások szövege.

Az alábbi eszközöket fogom a fejlesztéshez használni:

- **Dokumentáláshoz doxygen-t**
- **Teszteléshez a gtest_lite ellenőrző csomagot**
- **Memória kezelésre a memtrace csomagot**

Részletezés

Adatok beolvasása indításkor:

- **OO** változások: osztályok felhasználásával könnyebb, logikusabb lesz a tárolás és az egyes logikák megvalósítása, az egybezárásal egyedi viselkedést tudok majd végrehajtani a különböző osztályokon amik nagy mértékben megkönnyíti a felhasználásukat.
- **Háttérfolyamat**, ami **biztosítja**, hogy a program indításakor **az előző munkamenetben mentett vagy a felhasználó által megadott/kívánt adatokkal töltődjön fel.**
- **A Prog1-es alapokat továbbfejlesztettem egy rugalmasabb, kulcs-érték alapú adatszerkezetre.**
- Ez két fájlt jelent, melyek szerkezete az alábbi:

// Tanulók

Nev;Neptun_kod;Eloadas_csoport;Gyakorlati_csoport;Hianyzasok;ZH1;ZH2;ZH3;ZH4;ZH5;NZH_pont;Vizsga_pont

Kiss Anna;AABBCC;1;G01;2;15.50;18.00;0.00;0.00;0.00;35.00;42.00

Nagy Béla;CCDDEE;2;G03;0;19.00;20.00;18.50;15.00;0.00;40.00;50.00

Kovács Éva;FFGGHH;1;G02;5;12.50;10.00;0.00;0.00;0.00;28.50;30.00

// Oktatói

Nev;Csoport_1;Csoport_2;Csoport_3;Csoport_4;Csoport_5;Csoport_6;Csoport_7;Csoport_8;Csoport_9;Csoport_10

Dr. Tóth Pál;G01;G02;G05;;;;;

Prof. Kiss Rita;G03;G04;L01;;;;;

Szabó Gábor;G06;G07;G08;G09;L02;L03;;;;

- Ebben a formátumban a felhasználó megadhatja a saját használni kívánt adatait.
- **Ha ezek a fájlok nem léteznek, a program létrehozza őket üresen.**
- **Hibás vagy nem megfelelő adat szerkezetnél a program konzolon közli a hibát majd egy enter után leáll.**

Főmenü:

- A következő egy minta ami nem a végleges megvalósítást tükrözi, de tartalmazza azokat a fő elemeket amiket meg kell jeleníteni, hogy a felhasználó tudjon navigálni a főmenüből, ez a többi ábrára is igaz.

```
=====
|          HALLGATÓI NYILVÁNTARTÁS          |
=====

Válasszon menüpontot a számozott opciók közül:

[1] Listázás (Összes adat megjelenítése)
[2] Új adat felvétele (Hallgató/Oktató)
[3] Információ szerkesztése / törlése (CRUD műveletek)
[4] Fájlkezelés (Mentés/Betöltés)
-----
[0] Kilépés a programból

Adja meg a választott számot:
```

- Ahogy a példa is mutatja, a felhasználó számok beírásával tud navigálni az egyes almenük között. Ez azt jelenti, hogy az adott szám beütése és egy Enter után a program tovább lép a következő almenübe.
- A program le is állítható a megfelelő szám beírásával.
- Ezenkívül** minden almenüből vissza **lehet lépni** a főmenübe a megfelelő szám **megadásával**.
- A számok mellett látható az egyes menük neve és egy rövid leírás.

Listázás:

- Miután a felhasználó kiválasztotta a listázás almenüpontot, újabb menü fogadja.

```
=====
|               LISTÁZÁS MENÜ               |
=====

Válasszon egy listázási / elemzési opciót:

--- ALAP LISTÁK ---
[11] Összes hallgatói rekord listázása
[12] Összes oktatói rekord listázása
[13] Hallgatók listázása csoport szerint (pl. G01)

--- SZŰRÉS ÉS PÓTLÁS ---
[21] PZH-ra kötelezett hallgatók listája (Pót-ZH)
[22] Pótlásra kötelezett hallgatók listája (Nagy házi, labor)

--- RANGSOROLÁS ÉS STATISZTIKA ---
[31] Hallgatói rangsor (Összpontszám szerint, csökkenő sorrendben)
[32] Legtöbbet hiányzó hallgatók listája
[33] Csoportok összehasonlítása (Átlagpontszám szerint)

-----
[9]  Vissza a főmenübe

Adja meg a választott számot:
```

- A minta ismét szemléltetés jellegű.
- Itt a felhasználó kiválaszthatja, hogy melyik adattömböt szeretné látni, illetve hogy azt milyen szűrőkkel.
- **A szűrők előre definiáltak, a felhasználónak nincs lehetősége saját, egyéni szűrési lehetőséget megadnia.**
- Miután a felhasználó választott egy menüpontot egy egyszerű táblázat fogadja, **a táblázat nem alkalmaz** lapozást/lapozható megjelenítést.

HALLGATÓI RANGSOR - ÖSSZPONTSZÁM ALAPJÁN						
Hely	Név (30)	Neptun	Gyak.Cs.	Összpont	Értékelés	
1.	Nagy Béla	CCDDEE	G03	148	Kiváló	
2.	Kiss Anna	AABBCC	G01	111	Jeles	
3.	Kovács Éva	FFGGHH	G02	81	Elégtelen	
...	...	...	...	...	...	

Új adat felvétele:

- ismét a megfelelő almenü kiválasztása után az alábbi menüpontok jelennek meg:

```
=====
|      ÚJ ADAT FELVÉTELE MENÜ      |
=====

Milyen típusú új rekordot szeretne felvenni?

[1]  Új HALLGATÓ felvétele
[2]  Új OKTATÓ felvétele
-----
[9]  Vissza a főmenübe

Adja meg a választott számot:
```

- Ezután a felhasználónak meg kell adni az adatokat a választott táblához.
- Amennyiben a kötelező adatok közül bármelyik is üres, a program nem adja hozzá az új sort.**
- Ugyanez történik akkor is, ha a felhasználó nem megfelelő bemenetet ad.**
- Például típus probléma lép fel, vagy túl hosszú az általa megadott adat.**
- Ezeket a hibákat a program jelzi szöveges üzenet formájában a konzolon a felhasználónak.**
- Ezután visszalépés történik a főmenübe.

```
--- ÚJ HALLGATÓ FELVÉTELE ---
```

Kérem adja meg a hallgató adatait:

Név (max. 50 karakter): **Kiss Petra**

Neptun kód (6 karakter): **GH780P**

Előadás csoport (szám): **3**

Gyakorlati csoport (pl. G01): **L05**

Kérem adja meg a kezdeti eredményeket:

Hiányzások száma: **0**

ZH1 pont (pl. 18.5): **18.5**

NZH pont: **45.0**

Vizsga pont: **0.0**

[A ZH2, ZH3, ZH4, ZH5 pontok alapértelmezetten 0.0-ra állítva.]

A hallgatói rekord sikeresen felvéve a memóriába!

Nyomjon ENTER-t a folytatáshoz...

Megvalósítás

Adatszerkezet:

- Szöveges csv formátumú fájl.
- Minden sor tetszőleges számú kulcs-értékpárok sorozata, emiatt lehetséges az, hogy a sorokban lévő elemeket ne szigorú sorrendben kelljen beolvasni.

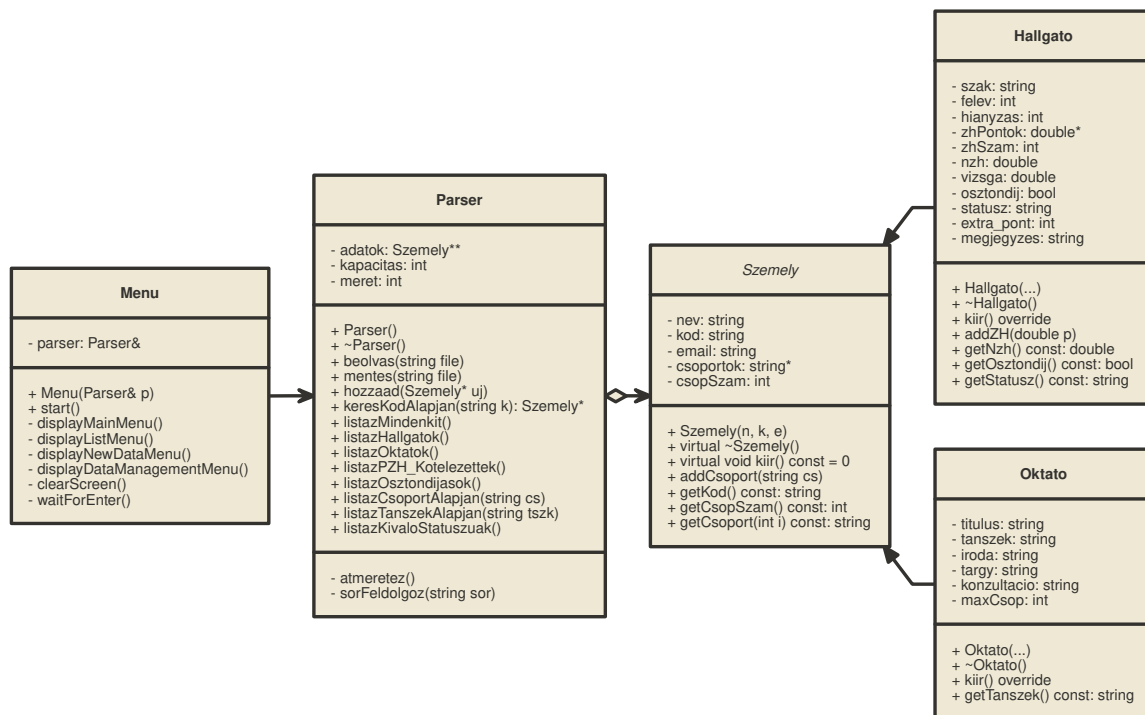
Oktatói rekord - Változó számú csoporttal

Tipus:Oktato;Nev:Dr. Tóth Pál;Kod:TP01;Titulus:PhD;Tanszek:IIT;Email:toth.pal@uni-pannon.hu;Iroda:I-123;Targy:Programozás
1;Csoport:G01;Csoport:G02;Csoport:G05;Max_Csoport:10

Hallgatói rekord - Változó számú ZH pontszámmal

Tipus:Hallgato;Nev:Kiss
Anna;Kod:AABBCC;Szak:Mérnökinformatikus;Felev:2;Email:k.anna@stud.uni.hu;Csoport:G01;Hi
ianyazas:2;ZH:15.5;ZH:18.0;ZH:0.0;ZH:0.0;ZH:0.0;NZH:33.5;Vizsga:42.0;Osztondij:Igen

Objektumorientált modell:



Szemely (Absztrakt őssosztály)

- **Közös attribútumok:** Tárolja minden személy alapadatait: név, Neptun-kód, e-mail cím.
- **Csoportkezelés:** Egy dinamikus string* tömbben tárolja a csoporttagságokat, lehetővé téve, hogy egy személy tetszőleges számú csoporthoz tartozzon.
- **Polimorfizmus:** Definiálja a virtuális destruktort és a tiszta virtuális kiir() metódust, biztosítva a helyes memóriafelszabadítást és az egyedi adatközlést.

Hallgato (Szemely osztályból származtatott)

- **Tanulmányi adatok:** Tárolja a hallgató szakát, aktuális félévét és az ösztöndíj státuszát.
- **ZH menedzsment:** Egy dinamikus double* tömbben gyűjti a pontszámokat. A kulcs-értékpáros beolvasás révén tetszőleges számú részeredményt képes kezelni.
- **logika:** Kiszámolja az összesített NZH pontszámot, figyeli a hiányzásokat, és ezek alapján megállapítja a vizsgára bocsáthatóságot vagy a PZH kötelezettséget.

Oktato (Szemely osztályból származtatott)

- Tárolja az oktató titulusát (pl. PhD), tanszékét és az irodájának azonosítóját.
- Kezeli a tanított tárgyat és a rögzített konzultációs időpontokat.
- Nyilvántartja a maximálisan vállalható csoportok számát (maxCsop).

Parser (Adatkezelő motor)

- **Fájlkezelés:** Felelős a perzisztens adattárolásért; indításkor betölti a CSV fájlt, kilépéskor pedig elmenti az aktuális állapotot.
- **Dinamikus memóriakezelés:** Egy Szemely** (mutatókra mutató mutató) tömbben tárolja az objektumokat. Ha a tömb megtelik, az atmeretez() metódus automatikusan, a kapacitás megduplázásával növeli a tárhelyet.
- **Rugalmas feldolgozás:** A sorFeldolgoz metódus kulcs-értékpárok alapján elemzi a bemenetet, így a mezők sorrendje tetszőleges lehet, az opcionális adatok hiánya pedig nem okoz hibát.
- **Lekérdezések:** Tartalmazza a szűrési logikákat (pl. keresés kód alapján, ösztöndíjasok listázása, tanszéki statisztikák).

Menu (Vezérlő interfész)

- **Interakció:** Kezeli a konzolos megjelenítést, kiírja a számozott menüpontokat és validálja a felhasználói bevitelt.
- **Vezérlés:** A start() metódus egy fő ciklusban tartja a programot a kilépési opció kiválasztásáért.
- **Adatfelvétel:** bekéri az új rekordok adatait, példányosítja a megfelelő osztályt (Hallgato vagy Oktato), majd átadja a Parser-nek tárolásra.

Algoritmusok:

1. Dinamikus tömb bővítése (Parser::atmeretez)

Ez a függvény biztosítja, hogy ne fogyjon el a hely az adatok tárolásakor.

ALGORITMUS atmeretez:

1. ÚJ_KAPACITÁS = JELENLEGI_KAPACITÁS * 2 // ez hosszútávon lehet optimálisabb
2. ÚJ_TÖMB = Memórafoglalás (Szemely* [ÚJ_KAPACITÁS])
3. CIKLUS i = 0-tól MERET-ig:
 - a. ÚJ_TÖMB[i] = RÉGI_TÖMB[i] (Mutatók átmásolása)
4. RÉGI_TÖMB felszabadítása (delete[] adatok)
5. ADATOK mutató átállítása az ÚJ_TÖMB-re
6. JELENLEGI_KAPACITÁS frissítése

VÉGE

2. Rugalmas sorfeldolgozás (Parser::sorFeldolgoz)

Ez kezeli a kulcs-értékpárokat és a változó számú mezőt.

ALGORITMUS sorFeldolgoz(sor):

1. DARABOK = Sor felvágása ';' mentén
2. TÍPUS = DARABOK[0]-ból kinyert érték (pl. "Hallgato")
3. OBJEKTUM = Új példány létrehozása a TÍPUS alapján (new)
4. CIKLUS minden DARAB-ra 1-től a végéig:
 - a. KULCS, ÉRTÉK = DARAB kettévágása ':' mentén
 - b. HA KULCS == "Nev": OBJEKTUM.setNev(ÉRTÉK)
 - c. HA KULCS == "ZH": OBJEKTUM.addZH(ÉRTÉK)
 - d. HA KULCS == "Csoport": OBJEKTUM.addCsoport(ÉRTÉK)... (többi mező)
5. hozzáad(OBJEKTUM) meghívása (eltárolás a tömbben)

VÉGE

3. Polimorf listázás (Parser::listazMindenkit)

Ez mutatja meg az objektumorientáltság erejét: egyetlen hívás, de mindenki mást ír ki.

ALGORITMUS listazMindenkit:

1. HA MERET == 0: Kiír ("Az adatbázis üres.")
2. CIKLUS i = 0-tól MERET-ig:
 - a. ADATOK[i].kiir()
// Itt a C++ a háttérben eldönti, hogy a
// Hallgato::kiir() vagy az Oktato::kiir() fusson le.
3. Kiír ("Összesen: [MERET] rekord.")

VÉGE